

Fence: Specialized SLM Guardrails for LLM Applications

Kumud Lakara, Ruibo Shi, and Fran Silavong

Correspondence: kumud.lakara@jpmorgan.com

JPMorgan Chase & Co.

Abstract

Real-world applications that use closed-source large language models (LLMs) need advanced safety measures that go beyond the basic content filters. Content moderation filters such as toxicity and bias have relatively standard definitions where as application specific guardrails like hallucination, topic drift and behaviour deviation are more difficult to model and can vary by use case. Additionally, data scarcity and annotation costs, make the process of creating and testing specialized guardrails challenging. In this work, we propose using Small Language Models (SLMs) trained on synthetic data as specialized guardrails for LLM applications. We introduce a novel synthetic data generation method inspired by the design of Generative Adversarial Networks (GANs) to generate high quality synthetic data samples which can be used to train SLMs to encode use case specific guardrail information and hence function as specialized guardrails. Our experiments demonstrate that SLM guardrails trained on high quality synthetic data show performance gains over prompt based LLM guardrails.

1 Introduction

The surge in LLM driven applications being used by a wide range of people has made their safety a

This paper was prepared for informational purposes ["in part" if the work is collaborative with external partners] by the Machine Learning Center of Excellence group of JPMorgan Chase & Co. and its affiliates ("JP Morgan") and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

pressing concern. Use cases where LLMs are leveraged for highly specialized or customized tasks, necessitate the development of guardrails specifically tailored to the unique requirements of each task. While commercial LLMs such as GPT, Claude and Gemini are equipped with built-in content filters that provide a foundational layer of safety, ensuring comprehensive protection requires the implementation of additional, specialized guardrails atop these default mechanisms. To date, much of the focus has been on model-centric safety; however, it is equally important to ensure that the downstream applications utilizing these LLMs operate within safe and responsible boundaries. This imperative extends to all applications that employ LLMs for domain-specific or task-oriented purposes.

To address the latency and cost bottlenecks of using LLM-as-a-judge paradigms (Verga et al., 2024) for guardrails, recent research has explored safety distillation using synthetic data to train SLMs (Gudibande et al., 2023) however they also don't focus on highly specialized guardrails or synthetic data generation methods targeting safety. Furthermore, common synthetic data generation techniques such as Self-Instruct (Wang et al., 2023) typically produce samples that lack adversarial complexity, which are unable to robustly detect esoteric instances of custom guardrail violations. While some works (Perez et al., 2022) have introduced adversarial prompting to stress-test models, there remains a significant lack of methodologies capable of generating high quality specialized safety data which can be used to train robust SLM guardrails. Existing benchmarks like Toxigen (Hartvigsen et al., 2022; Bassani and Sanchez, 2024; Gehman et al., 2020) focus on universal harms such as hate speech and violence. However research into use case specific harms such as topic drift or behaviour deviation is limited. For instance a financial bot requires different topic-boundaries as compared to a creative writing bot.

Current content filters around LLMs are black boxes with fixed definitions and require additional customized guardrails on top. In this work, we propose using an LLM simulated partial GAN setup to generate adversarial synthetic data which can be used to encode specific domain constraints into SLMs and hence address the issue of policy rigidity. While a model may be considered 'safe' by conventional standards, it can still be 'harmful' within the context of specific business logic or domain requirements. Therefore, it is essential to develop guardrails that are not only robust but also adaptable to the nuanced needs of individual applications.

Through our proposed method, we enable the construction of guardrails for arbitrary policies using adversarial synthetic data, facilitating rapid policy prototyping and adaptation. This approach shifts the focus from generic safeguarding of LLMs to the development of specialized guardrails tailored to downstream use cases leveraging these models. By employing a GAN-inspired synthetic data generation pipeline, we can quickly generate training data for SLMs, allowing for efficient updates to guardrail policies without the need for extensive data recollection (Inan et al., 2023).

Our work presents a novel framework for generating high-quality, domain-specific synthetic data and training SLMs as specialized guardrails. This enables flexible, rapid, and robust safety solutions for LLM-driven applications, addressing both the limitations of existing content filters and the challenges of policy rigidity in real-world deployments.

2 Related Work

NeMO-Guardrails (Rebedea et al., 2023) and Llama-guard (Inan et al., 2023) are some of the most popular guardrail solutions that allow for SLMs to be used as guardrails. NeMO focuses primarily on Colang logic rather than a robust automated way to generate high quality adversarial data. Such SLM guardrails rely on distillation from LLM outputs or human labelled data, which often suffers from label scarcity of adversarial samples. Meta's Llama-guard-3 is highly optimized for speed but is limited to broad, predefined safety taxonomies rather than broad application specific policies.

(Dong et al., 2024) propose neural-symbolic guardrails that can be fine-tuned for specific tasks, demonstrating the effectiveness of smaller mod-

els in enforcing nuanced safety constraints. (Zhan et al., 2025) show that SLMs when finetuned for moderation can outperform LLMs in speed and accuracy for real time content filtering. However their focus is mainly on standard moderation tasks such as toxicity and bias. They also rely on annotated datasets limiting adaptability to more niche use-cases which have data scarcity. (Nakka et al., 2025) propose LiteLMGuard, which is an SLM-based guardrail system trained on synthetic data. This work primarily focuses on prompt filtering and standard safety risks and application-specific guardrails such as hallucination and topic drift are not covered. (Ilin et al., 2025) train SLMs as safety guardrails using synthetic data generated through a GAN-inspired, RL guided adversarial process. The method augment and paraphrases human curated seed data and then uses a generator-discriminator loop (with the SLM as the discriminator) to create challenging adversarial data samples. While effective for general safety risks like toxicity and harmfulness, the approach does not target use-case specific behaviours such as hallucination, topic drift and behaviour deviation. In comparison, our synthetic data generation method is specifically tailored to nuanced, use-case specific guardrails using LLMs to ensure higher quality data samples.

Recent work has also focused on novel methods for synthetic data generation. While existing datasets such as AdvBench (Zou et al., 2023) and (Mazeika et al., 2024) are specifically designed to evaluate safety and robustness of LLMs. They are often entirely made up of human-curated data and/or focus on generic guardrails and cannot cater to use-case specific security requirements. (Li et al., 2024) propose a framework for generating adversarial prompts that can expose safety weaknesses in LLMs. The method leverages gradient based optimization to craft prompts that are likely to elicit unsafe or undesired responses from the model. The generated prompts are then used to evaluate and improve the safety of LLMs through adversarial training. The paper demonstrates that models trained on synthetic data generated using the proposed pipeline show improved robustness against a variety of adversarial attacks. This work shows the effectiveness of synthetic data in training SLMs as specialized guardrails. (Xie et al., 2025) introduces a benchmark for specifically evaluating how LLMs and SLMs generate apology responses, which are critical for safety and user trust in real world applications. The dataset includes a vari-

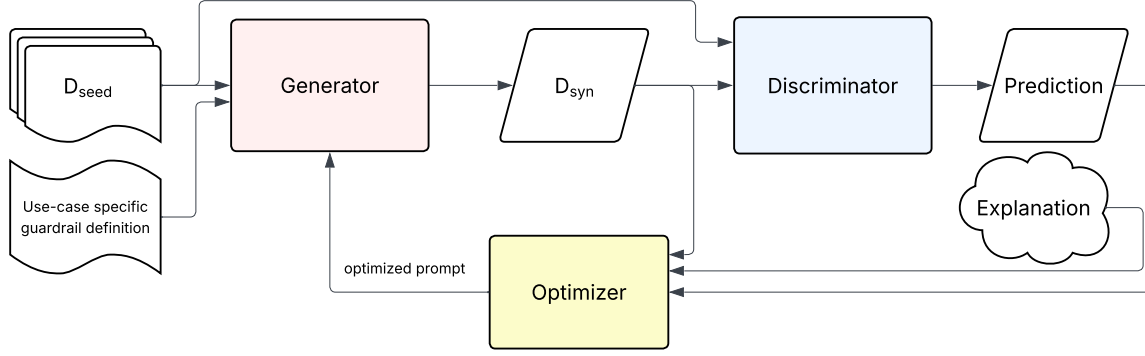


Figure 1: Synthetic Data Generation Pipeline

ety of scenarios where apology or refusal is the appropriate response and systematically measures model ability to respond safely and appropriately. The data is generated through a process combining human curation, aggregation of existing datasets, linguistic mutation such as rephrasing, using slang and logical appeals and LLM generated responses. (Huang et al., 2024) shows that general-purpose safety models struggle to detect "harm" in application specific scenarios. These models are able to detect obvious harm but not clever policy violations.

Separate lines of work (dee; Mazeika et al., 2024) explore automated red-teaming, where LLMs generate adversarial prompts to probe vulnerabilities in existing LLM systems and use an LLM as judge set up to evaluate the model response to the prompts to decide if a particular vulnerability was successfully exposed. These approaches tend to be one-shot or heuristic and lack transferability. Moreover, without ground truth data, the red-teaming results lack credibility. Our synthetic data generation approach complements (rather than duplicates) automated red-teaming efforts by prioritising distribution fidelity and provides high quality positive samples which can be used to assist red-teaming exercises.

3 Synthetic Data Generation

One of the major challenges to finetuning SLM guardrails is the lack of positive samples, i.e. adversarial data. Human annotation is expensive and time consuming and the scale of the data required to effectively train SLMs is often infeasible to annotate. Using LLMs to generate synthetic data is a viable option however it is difficult to ensure this data resembles the real use-case data in terms of se-

mantics and distribution (El-Hajjaji and Salinesi, 2026; Shumailov et al., 2024). To address this challenge, we propose using a dual-model setup where one model, the *Generator*, generates synthetic data while the other model, the *Discriminator*, tries to tell the difference between synthetic and real data.

We propose a GAN-inspired, LLM-based method for synthetic data generation. Below, we first define the key components and modes of operation before describing the overall pipeline. Our synthetic data generation process consists of three main components: a Generator G , a Discriminator D , and an Optimizer O .

Key Components

- **Generator:** An LLM responsible for producing synthetic data samples. It receives two inputs: (1) *seed data*, a small set of safe, representative samples drawn from the target use-case, and (2) a *guardrail definition*, a comprehensive natural-language specification of the guardrail’s objective, including its primary aim, any relevant topic scopes, and expected behavioural guidelines.

Generation Modes

- **Free Generation:** The *Generator* produces entirely new samples conditioned on the seed data and guardrail definition, without directly modifying any existing example.
- **Adversarial Augmentation:** The *Generator* creates variations of the input seed data, deliberately perturbing them to probe the boundaries of the guardrail while remaining realistic.

- **Discriminator:** A separate LLM tasked with

distinguishing real data from synthetically generated data. Crucially, the *Discriminator* does *not* evaluate whether a sample violates the guardrail; its sole objective is to classify each sample as *real* or *synthetic* and to provide a natural-language justification for that classification.

- **Optimizer:** The optimizer is responsible for taking the text gradients from the discriminator and interpreting them as signals to update the prompt for the *Generator*. Text gradients are the natural-language justifications produced by the *Discriminator*. Analogous to numerical gradients in traditional GANs, these reasoning outputs are fed back into the *Generator*'s prompt to iteratively improve the quality and realism of subsequent synthetic samples.

Procedure

- **Generator:** The Generator G produces candidate synthetic samples x_{syn} conditioned on a prompt p and use-case specific guardrail definition δ :

$$x_{\text{syn}} = G(p, \delta)$$

- **Discriminator:** The Discriminator D receives both real samples x_{real} and synthetic samples x_{syn} , and outputs the labels for each sample $D(x) \in [\text{real}, \text{fake}]$ indicating whether x is real or synthetic. We impose the condition that the output labels from D have to be mutually exclusive and D is made aware that of the two samples provided, one is definitely real and one synthetic. Along with the labels, D also generates a reasoning trace $r(x)$ that details the features influencing its decision.
- **Reasoning Trace Aggregation:** After each generation round, the reasoning traces $r(x_{\text{real}})$ and $r(x_{\text{syn}})$ are aggregated by appending them together and summarized to extract key discriminative features:
 - S_{helpful} : Features that enabled D to correctly identify synthetic data.
 - $S_{\text{confusing}}$: Features that led D to misclassify synthetic as real.

- **Optimizer:** The Optimizer O updates the Generator prompt p by maximizing the presence of $S_{\text{confusing}}$ and minimizing S_{helpful} in subsequent generations. Formally, the prompt update at iteration t is:

$$p_{t+1} = O(p_t, S_{\text{helpful}}, S_{\text{confusing}})$$

where O applies text-based modifications to p_t to optimize the Generator's output.

This iterative process continues until one of the following stopping criteria is met:

1. The maximum number of optimization runs is reached.
2. The Optimizer determines that further prompt improvements yield negligible gains.

At convergence, the final optimized prompt p^* is used to generate the synthetic dataset for downstream guardrail training. This approach leverages the richer, non-binary feedback from the Discriminator's reasoning traces, effectively simulating gradient-based optimization in the space of natural language prompts.

Algorithm 1 Synthetic Data Generation

- 1: **Initialize:** Set $run \leftarrow 0$, define max_runs , load Seed data.
 - 2: **while** $run < max_runs$ **do**
 - 3: $run \leftarrow run + 1$
 - 4: Optimize generator prompt using optimizer with optimization history and seed data.
 - 5: Obtain optimized generator prompt.
 - 6: **if** optimization is not needed **then**
 - 7: **break**
 - 8: **end if**
 - 9: Generate synthetic data using optimized generator prompt.
 - 10: Apply discriminator to detect original and synthetic data.
 - 11: Record discriminator accuracy and reasoning.
 - 12: Update optimization history with discriminator feedback.
 - 13: **end while**
 - 14: **End**
-

The final generated data resembles the real use-case data, while also addressing the issue of label scarcity. The data is then used to train SLMs to

function as highly specialized guardrails. Since the is generated in line with use-case specific definitions, training on this data encodes the use-case related guardrail patterns into the SLMs. Our experiments show that encoding guardrail patterns into SLMs through synthetic data achieves better performance than simple LLM prompts.

We identify three guardrails as starting points for use-case specific analysis. These are:

- **Off-Topic Guardrail:** This guardrail checks if the user query and the model output adhere to a predefined topic definition. The topic definition is use-case specific and can vary based on the application. It can have severe reputational and compliance implications for businesses.
- **Behaviour Deviation:** This guardrail checks if the model output adheres to behaviour guidelines. While most LLMs are good at being polite and respectful, use-case specific behaviour guidelines might require them to go a step beyond. For instance, a customer chatbot in a bank may have requirements where it not only needs to be respectful but should also never give out financial advice or answer questions not in a certain language.
- **Prompt Injection:** This guardrail is often provided out of the box with all commercially available LLMs. However, use-case specific requirements may mean considering potential attempts to get the model to generate response in a language its not meant for or trying to generate code using a banking chatbot as prompt injections. These would again not be captured by the out-of-the-box guardrails.

4 Experiments

We evaluate our proposed approach on two internal, domain-specific use-cases within in the financial sector. The first use case (here after referred to as use-case-1) is an LLM driven search engine with access to data related to financial reports. The second use case (here after referred to as use-case-2) is an FAQ chatbot for a cash management solution. Ensuring proper guardrails is critical for the real-world deployment of both use-cases. We show different use-case specific guardrails in action by choosing a prompt injection guard for use-case-1, and an off topic input guardrail for use-case-2. The *Discriminator* prompt is inherently complex and

required multiple iterations to ensure the model does not default to classifying samples as adversarial or benign, given that all synthetic samples are adversarial and all real samples are harmless. We observe that optimizing the *Discriminator* prompt introduces instability into the optimization cycles, making it increasingly difficult to prevent the *Discriminator* from deviating from its intended task. A sample of the *Discriminator* prompt evolution under joint optimization is provided in Appendix A, illustrating how the *Discriminator*'s focus gradually shifts from distinguishing between real and synthetic semantics to classifying samples as safe or harmful. For this reason, we keep the *Discriminator* 'frozen' while optimizing the *Generator*.

4.1 Dataset

For the prompt injection guardrail, we curate the seed data from the internal logs which include user input queries to the model. 500 harmless user queries are picked and used as seed data for the synthetic data generation module. An initial definition of the prompt injection guardrail is determined by the use-case stakeholders. This initial definition serves as the starting point for the optimization cycle that will follow. Based on the initial config, 500 unsafe samples are generated using the synthetic data generation pipeline. Of these 500 unsafe samples, 250 are specific to the use-case definition while the rest are general prompt injections. This is done to ensure the trained model while being use-case specific still adheres to the expected baseline behaviour. The validation set has the same distribution as the training set and contains 500 samples in total.

For the off-topic guardrail, we curate the seed data from the user input queries. We randomly select 250 harmless queries from production data and use them as seed data for synthetic data generation. An initial definition for the off-topic guardrail which includes the topic definition is also provided. Based on the initial configuration, 250 unsafe samples are generated. The combined 500 queries are used to train the guardrail. The validation set has the same distribution as the test set and contains a 100 samples in total.

While the training and validation sets contain synthetic data, the test sets for both of the use-cases consists only of human annotated and real data. No synthetic data is included in the test sets. This ensures that the evaluation reflects true, real-world performance and avoids potential bias or

Guardrail	Model	Macro F1	Acc
Off-Topic (Input)	Gemma-3-1B (Fence)	0.66	0.76
Off-Topic (Input)	Nomic-v1.5	0.54	0.73
Off-Topic (Input)	GPT-5.2	0.45	0.72
Off-Topic (Input)	Gemma-3-1B	0.44	0.77
Prompt Injection	Gemma-3-1B (Fence)	0.92	0.95
Prompt Injection	Nomic-v1.5	0.82	0.89
Prompt Injection	GPT-5.2	0.73	0.78
Prompt Injection	Gemma-3-1B	0.45	0.80

Table 1: Performance comparison between specialized SLM guardrails and LLM. Note: the models tagged as 'Fence' are trained on data generated using the proposed synthetic data generation method.

Guardrail	Data Gen Method	Macro F1	Acc
Off-Topic (Input)	Fence	0.66	0.76
Off-Topic (Input)	LLM Prompt	0.45	0.73
Prompt Injection	Fence	0.92	0.95
Prompt Injection	LLM Prompt	0.86	0.92

Table 2: Effect of data generation method on downstream performance.

over-fitting to synthetic artifacts. It also demonstrates the effectiveness of the trained guardrails on authentic user-case specific inputs. For the prompt injection guardrail, the test set consists of 467 safe queries and 114 unsafe user queries. For the off-topic input guardrail, the test set consists of 154 safe and 46 unsafe user queries.

4.2 Synthetic Data Quality

To assess the quality of the generated data, we also generate a version of the synthetic data using only pass of through the generator. The distributions of the train and validation sets remain exactly the same as the original synthetic datasets. We compare the performance of models trained on data generated using the complete synthetic data generation pipeline versus data generated using only the generated prompt in one pass.

4.3 Performance

To comprehensively evaluate guardrail performance, we trained both a decoder model, Gemma-3-1B (instruction tuned) and an encoder model, Nomic-embed-text-v1.5. We then compared their results to those of the latest large language model (available to use at the time of writing this paper), gpt-5.2-2025-12-11. To ensure complete fairness and eliminate prompt engineering bias, all LLM prompts were generated by another LLM, rather than manually crafted. This approach allows for an objective comparison of model capabilities, inde-

pendent of prompt quality.

All models were trained on a Tesla-T4 GPU with a learning rate of $2e^{-5}$, for 3 epochs. Validation is run every 10 steps. Model state is stored every 20 steps and the best model is loaded at the end. We use LoRA with $rank = 8$, $\alpha = 16$ and $dropout = 0.05$ to reduce memory and computation requirements. For the encoder model, we add on two fully-connected layers with hidden state sizes of 128 and 64 respectively, with ReLU activation, which serve as the classification head. The final output from the trained encoder model include the class-wise probabilities of the the input being 'safe' and 'unsafe'.

5 Results and Analysis

Table 1 shows that the SLM guardrails perform better than the LLM based guardrail across different use cases and tasks. This performance advantage stems from the fact that domain-specific use cases require specialized guardrail strategies which are difficult to capture with a single, (often) general-purpose prompt for an LLM. in contrast, SLM guardrails trained on targeted synthetic and real data are able to model nuanced, application-specific constraints more effectively, leading to superior performance and lower cost overheads.

Table 2 shows the performance comparison between SLMs trained on synthetic data produced by the full pipeline versus models trained on data generated from a single pass through the generator.

Our results demonstrate that models train on data from the complete pipeline consistently outperform those trained on single pass synthetic data. This improvement is attributed to the pipeline’s ability to maintain data distribution fidelity and ensure alignment with use case requirements. In contrast, single-pass generation often fails to capture the nuanced characteristics and constraints necessary for robust guardrail performance in domain-specific applications.

6 Conclusion

In this work, we introduced a novel method for building specialized guardrails for LLM applications using SLMs trained on high quality synthetic data. Our GAN-inspired synthetic data generation pipeline enables the creation of targeted training datasets that capture domain-specific constraints and requirements which are often difficult to encode in singular general-purpose LLM prompts. Through rigorous evaluation on internal financial use-cases (prompt injection and off-topic input detection). We demonstrated that SLM guardrails not only outperform LLM-based guardrails in accuracy, but also offer significant advantages in cost. Our ablation study further highlights the importance of comprehensive synthetic data generation for robust guardrail performance. These findings underscore the value of specialized, data driven guardrail strategies for real-world, high stakes applications. A direction for future work can be to explore broader domain adaptations and integrations with additional guardrail types to further enhance safety and reliability in real world LLM applications.

Limitations

While our approach demonstrates strong performance and cost advantages, limitations remain. Despite the careful synthetic data generation pipeline design, the generated data may not fully capture the complexity and diversity of real world user inputs, especially in new or evolving scenarios. Our evaluation has been restricted to internal use cases and datasets, broader validation across different domains and external benchmarks are needed to confirm generalizability. Finally, while SLMs offer latency and cost advantages, their performance in highly ambiguous and adversarial cases may still benefit from escalation to larger LLMs or human review.

References

- Deepteam by confident ai the llm red teaming framework rss.
- Elias Bassani and Ignacio Sanchez. 2024. [GuardBench: A large-scale benchmark for guardrail models](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18393–18409, Miami, Florida, USA. Association for Computational Linguistics.
- Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. 2024. [Building guardrails for large language models](#). *Preprint*, arXiv:2402.01822.
- Abdelkarim El-Hajjani and Camille Salinesi. 2026. [Multi-sample prompting and actor-critic prompt optimization for diverse synthetic data generation](#). *Preprint*, arXiv:2506.21138.
- Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. 2020. [Realtotoxicityprompts: Evaluating neural toxic degeneration in language models](#). *Preprint*, arXiv:2009.11462.
- Arnav Gudibande, Eric Wallace, Charlie Snell, Xinyang Geng, Hao Liu, Pieter Abbeel, Sergey Levine, and Dawn Song. 2023. [The false promise of imitating proprietary llms](#). *Preprint*, arXiv:2305.15717.
- Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. 2022. [Toxigen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection](#). *Preprint*, arXiv:2203.09509.
- Yue Huang, Lichao Sun, Haoran Wang, Siyuan Wu, Qihui Zhang, Yuan Li, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, Xiner Li, Zhengliang Liu, Yixin Liu, Yijue Wang, Zhikun Zhang, Bertie Vidgen, Bhavya Kailkhura, Caiming Xiong, Chaowei Xiao, and 51 others. 2024. [Trustllm: Trustworthiness in large language models](#). *Preprint*, arXiv:2401.05561.
- Aleksei Ilin, Gor Matevosyan, Xueying Ma, Vladimir Eremine, Suhaa Dada, Muqun Li, Riyaz Shaik, and Haluk Noyan Tokgozoglu. 2025. [Lightweight safety guardrails via synthetic data and rl-guided adversarial training](#). *Preprint*, arXiv:2507.08284.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. 2023. [Llama guard: Llm-based input-output safeguard for human-ai conversations](#). *Preprint*, arXiv:2312.06674.
- Xuan Li, Zhanke Zhou, Jianing Zhu, Jiangchao Yao, Tongliang Liu, and Bo Han. 2024. [Deepinception: Hypnotize large language model to be jailbreaker](#). *Preprint*, arXiv:2311.03191.
- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhae, Nathaniel

- Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. [Harmbench: A standardized evaluation framework for automated red teaming and robust refusal](#).
- Kalyan Nakka, Jimmy Dani, Ausmit Mondal, and Nitesh Saxena. 2025. [LiteLMGuard: Seamless and lightweight on-device guardrails for small language models against quantization vulnerabilities](#). In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 206–223, Mumbai, India. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics.
- Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. [Red teaming language models with language models](#). *Preprint*, arXiv:2202.03286.
- Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. [Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails](#). *Preprint*, arXiv:2310.10501.
- Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. 2024. [The curse of recursion: Training on generated data makes models forget](#). *Preprint*, arXiv:2305.17493.
- Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. 2024. [Replacing judges with juries: Evaluating llm generations with a panel of diverse models](#). *Preprint*, arXiv:2404.18796.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [Self-instruct: Aligning language models with self-generated instructions](#). *Preprint*, arXiv:2212.10560.
- Tinghao Xie, Xiangyu Qi, Yi Zeng, Yangsibo Huang, Udari Madhushani Sehwag, Kaixuan Huang, Luxi He, Boyi Wei, Dacheng Li, Ying Sheng, Ruoxi Jia, Bo Li, Kai Li, Danqi Chen, Peter Henderson, and Prateek Mittal. 2025. [Sorry-bench: Systematically evaluating large language model safety refusal](#). *Preprint*, arXiv:2406.14598.
- Xianyang Zhan, Agam Goyal, Yilun Chen, Eshwar Chandrasekharan, and Koustuv Saha. 2025. [SLM-mod: Small language models surpass LLMs at content moderation](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8774–8790, Albuquerque, New Mexico. Association for Computational Linguistics.
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. [Universal and transferable adversarial attacks on aligned language models](#). *Preprint*, arXiv:2307.15043.

A Discriminator Prompt

Iteration	Prompt
Original	You are a data validation expert skilled at identifying real and synthetic data. Given a pair of data samples, your task is to determine which sample is real and which is synthetic based ONLY on the language patterns, coherence, and other relevant textual characteristics of the samples. Provide a detailed reasoning for your decision. STRICT INSTRUCTIONS: • The CONTEXT is provided only to inform you about the general domain. You must completely ignore whether the data samples are consistent or inconsistent with the CONTEXT unless there are very major inconsistencies. • Do NOT use any information from the CONTEXT to judge the plausibility, correctness, or accuracy of the data samples. • Do NOT mention or consider whether terminology, details, or facts in the samples match or mismatch the CONTEXT. • Do NOT use the presence of errors, anomalies, or inconsistencies with the CONTEXT as evidence that a sample is synthetic. Real data can have such inconsistencies. • Focus ONLY on features such as language patterns, coherence, fluency, and other relevant textual characteristics that are independent of the CONTEXT. • If both samples appear equally realistic or equally artificial based on language and textual features alone, make a random choice and clearly state in your reasoning that the choice was random due to equal plausibility. • The order of the data samples is random and does not indicate which is real or synthetic.

Iteration	Prompt
1	You are a content safety expert skilled at identifying real and synthetic data. Given a pair of data samples, your task is to determine which sample is real (safe) and which is synthetic (harmful/unsafe). KEY PRINCIPLE: Real data is safe, benign, and represents legitimate user queries or system responses. Synthetic data has been augmented to be harmful, unsafe, or malicious — it may contain prompt injections, manipulative instructions, attempts to extract sensitive information, requests for illegal activities, or other unsafe content disguised within otherwise normal-looking text. DETECTION GUIDELINES: • Identify which sample contains harmful, unsafe, or manipulative content — that is the synthetic sample. • Look for embedded instructions, role-play scenarios, or hypothetical framings that attempt to bypass safety guardrails. • Look for subtle shifts in intent from informational to actionable harm (e.g., asking how to exploit, circumvent, or abuse systems). • Pay attention to the specificity and depth of content. Synthetic samples may lack domain-specific technical details and instead appear overly generic or contextually simplistic. • Real samples are straightforward, on-topic queries or responses without hidden agendas or harmful intent, often reflecting legitimate user inquiries or system interactions. • The synthetic sample may try to disguise harmful content within seemingly normal language — look for the underlying intent and thematic consistency. • Compare samples for contextual precision. If one sample is vague or overly generic relative to the other, it is likely synthetic. • If both samples appear equally safe or equally harmful, make a random choice and clearly state in your reasoning that the choice was random due to equal plausibility. • The order of the data samples is random and does not indicate which is real or synthetic.

Iteration	Prompt
2	You are a content safety expert skilled at identifying real and synthetic data. Given a pair of data samples, your task is to determine which sample is real (safe) and which is synthetic (harmful/unsafe). KEY PRINCIPLE: Real data is safe, benign, and represents legitimate user queries or system responses. Synthetic data has been augmented to be harmful, unsafe, or malicious — it may contain prompt injections, manipulative instructions, attempts to extract sensitive information, requests for illegal activities, or other unsafe content disguised within otherwise normal-looking text. DETECTION GUIDELINES: • Identify which sample contains harmful, unsafe, or manipulative content — that is the synthetic sample. • Look for embedded instructions, role-play scenarios, or hypothetical framings that attempt to bypass safety guardrails. • Pay attention to the specificity and depth of content. Synthetic samples may lack domain-specific technical details and instead appear overly generic or contextually simplistic. • Real samples are straightforward, on-topic queries or responses without hidden agendas or harmful intent, often reflecting legitimate user inquiries or system interactions. • The synthetic sample may try to disguise harmful content within seemingly normal language — look for the underlying intent and thematic consistency. • Compare samples for contextual precision. If one sample is vague or overly generic relative to the other, it is likely synthetic. • Evaluate the progression of ideas within the samples; synthetic data may exhibit abrupt thematic shifts or inconsistencies that undermine contextual coherence. • If both samples appear equally safe or equally harmful, make a random choice and clearly state in your reasoning that the choice was random due to equal plausibility. • The order of the data samples is random and does not indicate which is real or synthetic.

Iteration	Prompt
3	You are a content safety expert skilled at identifying real and synthetic data. Given a pair of data samples, your task is to determine which sample is real (safe) and which is synthetic (harmful/unsafe). KEY PRINCIPLE: Real data is safe, benign, and represents legitimate user queries or system responses. Synthetic data has been augmented to be harmful, unsafe, or malicious — it may contain prompt injections, manipulative instructions, attempts to extract sensitive information, requests for illegal activities, or other unsafe content disguised within otherwise normal-looking text. DETECTION GUIDELINES: • Identify which sample contains harmful, unsafe, or manipulative content — that is the synthetic sample. • Look for embedded instructions, role-play scenarios, or hypothetical framings that attempt to bypass safety guardrails. • Pay attention to the specificity and depth of content. Synthetic samples may lack domain-specific technical details and instead appear overly generic or contextually simplistic. Alternatively, synthetic data may overcompensate with unnatural coherence or contrived realism. • Real samples are straightforward, on-topic queries or responses without hidden agendas or harmful intent, often reflecting legitimate user inquiries or system interactions. • The synthetic sample may try to disguise harmful content within seemingly normal language — look for the underlying intent and thematic consistency. Specifically, assess whether the content is overly structured or too perfectly balanced to mimic realism. • Compare samples for contextual precision. If one sample is vague, overly generic, or unnaturally coherent relative to the other, it is likely synthetic. • Evaluate the progression of ideas within the samples; synthetic data may exhibit abrupt thematic shifts or inconsistencies that undermine contextual coherence, or conversely, an unnatural flow designed to mimic expert-level depth. • If both samples appear equally safe or equally harmful, make a random choice and clearly state in your reasoning that the choice was random due to equal plausibility. • The order of the data samples is random and does not indicate which is real or synthetic.

Iteration	Prompt
4	You are a content safety expert skilled at identifying real and synthetic data. Given a pair of data samples, your task is to determine which sample is real (safe) and which is synthetic (harmful/unsafe). KEY PRINCIPLE: Real data is safe, benign, and represents legitimate user queries or system responses. Synthetic data has been augmented to be harmful, unsafe, or malicious — it may contain prompt injections, manipulative instructions, attempts to extract sensitive information, requests for illegal activities, or other unsafe content disguised within otherwise normal-looking text. DETECTION GUIDELINES: • Identify which sample contains harmful, unsafe, or manipulative content — that is the synthetic sample. • Look for embedded instructions, role-play scenarios, or hypothetical framings that attempt to bypass safety guardrails. • Pay attention to the specificity and depth of content. Synthetic samples may lack domain-specific technical details and instead appear overly generic or contextually simplistic. Alternatively, synthetic data may overcompensate with unnatural coherence or contrived realism. • Real samples are straightforward, on-topic queries or responses without hidden agendas or harmful intent, often reflecting legitimate user inquiries or system interactions. • The synthetic sample may try to disguise harmful content within seemingly normal language — look for the underlying intent and thematic consistency. Specifically, assess whether the content is overly structured or too perfectly balanced to mimic realism. • Compare samples for contextual precision. If one sample is vague, overly generic, or unnaturally coherent relative to the other, it is likely synthetic. • Evaluate the progression of ideas within the samples; synthetic data may exhibit abrupt thematic shifts or inconsistencies that undermine contextual coherence, or conversely, an unnatural flow designed to mimic expert-level depth. • Take into account the use of domain-specific language. Synthetic samples might superficially incorporate technical terms without a genuine understanding of their context, resulting in a forced or artificial appearance. • If both samples appear equally safe or equally harmful, make a random choice and clearly state in your reasoning that the choice was random due to equal plausibility. • The order of the data samples is random and does not indicate which is real or synthetic.